

Solving Sudoku Using Oscillatory Neural Networks

Stefan Porfir, Bram F. Haverkort, Federico Sbravati, Aida Todri-Sanial

NanoComputing Lab

Integrated Circuits

Electrical Engineering

Eindhoven University of Technology

Eindhoven, The Netherlands

Abstract—This paper explores the application of Oscillatory Neural Networks (ONNs) to solving Sudoku puzzles, presenting a biologically inspired approach based on phase synchronization. Each cell is represented by an oscillator whose phase encodes a digit, and the synchronization is governed by the Kuramoto model. The system dynamically evolves towards a valid solution by having the puzzle constraints encoded into the weight matrix of the network, and through a proposed novel phase mapping of the Sudoku digits. Experimental results show that ONNs achieve high performance for puzzles with moderate difficulty and outperform Hopfield Neural Networks, particularly in cases with up to 20 initially unknown values. Although the performance decreases with increased ambiguity, ONNs still produce correct solutions in some of the iterations, cases in which the baseline Hopfield Neural Network algorithm fails. The findings support ONNs as a viable alternative for solving constraint optimization problems and reinforce their relevance within emerging non-von Neumann computing paradigms.

I. INTRODUCTION

Since the rise of modern computing, the von Neumann architecture played an important role for shaping the way information is processed [1]. However, as computing demands grow, especially in the fields of Artificial Intelligence (AI), large-scale optimization and real-time data processing, the von Neumann Bottleneck seems to become increasingly evident [2]. This bottleneck refers to the separation between memory and processing units, which implies limitations on data throughput and energy efficiency. As a result, researchers began exploring non-von Neumann computing paradigms. To break free from these limitations of traditional architectures, current research is being developed in promising fields such as Neuromorphic Computing and Quantum Computing [3]. Neuromorphic approaches attempt to create energy efficient hardware by emulating the parallel and event-driven nature of the brain [4], [5], while Quantum Computing systems offer novel capabilities for probabilistic computation and optimization [6]. In this context, Oscillatory Neural Networks represent a novel and biologically inspired computing model that benefits from the dynamics of coupled oscillators to

solve constraint satisfaction and optimization problems while maintaining low-energy characteristics [5].

Oscillatory Neural Networks (ONNs) are an emerging computing paradigm for AI applications, which has gained attention due to its parallelism and low-power computing [7]. ONNs simulate the oscillatory behavior of real neural oscillations in the brain through coupled oscillators. The potential efficiency is based on the parallelism of the network, enabling it to perform multiple computations simultaneously [8]. The particularity presented by ONNs is the fact that the information is encoded in the phase relations between the oscillators present in the system, making parallel computation possible. Furthermore, the use of phase relations enables a larger spectrum for encoding, meaning that information could be encoded over the whole range of $[0, 2\pi)$. Having the whole phase circle available enables each oscillator to represent multiple logic states through distinct phase angles, breaking free from limitations of binary encoding, where information can only be encoded in either ON or OFF states. Moreover, the flexibility in encoding options further enforces the potential of low-power computing of ONNs, needing less resources for optimization tasks, especially multi-valued problems such as Sudoku, where each cell must take the value of one of several digits. The continuous nature of the phase circle could contribute to smoother system dynamics and faster convergence.

Sudoku is a widely recognized puzzle, where the challenge lays in filling a grid adhering to numerical constraints such that each row, column and subgrid contains all numbers exactly once [9]. Inherently, Sudoku can be formulated as a constraint satisfaction problem, where each digit assignment must meet unique constraints across the grid. Traditional approaches to solving such problems include backtracking [10] or more advanced techniques such as SAT solvers [11]. Oscillators have also been employed in convolutional neural network as attention layers to solve Sudoku [12], but such approaches use training instead of directly applying natural oscillator dynamics. Moreover, being a simple constraint satisfaction problem, solving Sudoku using ONNs has the potential to be related to the Ising model, where solving the puzzle through an ONN Weight Matrix is similar to finding a low-energy configuration that satisfies a set of constraints. This formulation opens the door to unconventional computation models, where the dynamics of physical or neuromorphic systems can

This work has received funding from the European Research Council ERC THERMODON project under grant agreement No. 101125031 and Dutch Research Council's AiNed Fellowship research program, AI-on-ONN project under grant agreement No. NGF.1607.22.016. (Corresponding authors: Bram F. Haverkort, b.f.haverkort@tue.nl; Federico Sbravati, f.sbravati@tue.nl; Aida Todri-Sanial, a.todri.sanial@tue.nl.)

1	2	3	4
3	4	1	2
4	3	2	1
2	1	4	3

Fig. 1. Completed 4×4 Sudoku grid.

be harnessed to solve combinatorial problems in a parallel and low-energy manner. The existing literature has tackled solving Sudoku through Artificial Neural Networks, such as Hopfield Neural Networks (HNN) [13], [14] or Photonic Spiking Neural Networks [15], [16], making it possible to have comparisons with the performance of the method proposed in this paper, namely, solving Sudoku using the free-dynamics of Oscillatory Neural Networks.

The goal of this paper is to implement a mapping for ONNs to solve the well-known Sudoku puzzle, by means of the natural Kuramoto dynamics of the system. Furthermore, the paper aims to provide comparisons of the results with a state-of-the-art Hopfield Network approach [13], [14].

II. BACKGROUND

The model used to govern the ONN phase dynamics and the synchronization of the oscillators is the Kuramoto Model [17], through the following equation:

$$\frac{d\theta_i}{dt} = \omega_i^{(0)} - \sum_{j=1}^N J_{ij} \sin(\theta_i - \theta_j), \quad (1)$$

where:

- θ_i is the phase of oscillator i (in radians),
- $\frac{d\theta_i}{dt}$ is the instantaneous rate of change of the phase (angular velocity),
- $\omega_i^{(0)}$ is the natural frequency¹ of oscillator i .
- J_{ij} is the coupling strength between oscillators i and j . The matrix is complex, and non-symmetric in the presented mapping in general.
- $\sin(\theta_i - \theta_j)$ captures the phase difference between oscillator i and oscillator j .

The numerical representation of the Kuramoto model has been implemented in Python and the numerical method used to solve the Kuramoto model is the Runge-Kutta adaptive solver of order 5 [18], [19]. The function simulates the Kuramoto synchronization model [20], and returns the phases of the oscillators after synchronization. The synchronization of the oscillators allows the oscillators to lock in the right phases for the corresponding digits.

III. METHODS

The methodology for solving Sudoku through natural dynamics is illustrated by means of a 4×4 Sudoku variant

¹The natural frequency $\omega_i^{(0)}$ is assumed to be the same for each oscillators, and is thus neglected in the simulations.

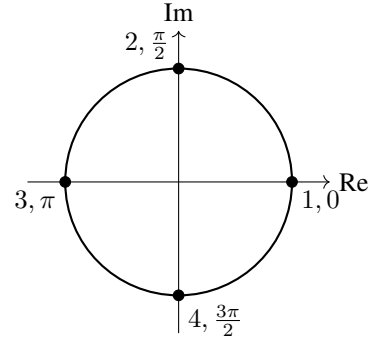


Fig. 2. Digits of a 4×4 Sudoku mapped onto the unit circle with their respective phase values.

depicted in Fig. 1. This simplified case allows for clearer explanation of the natural dynamics that solve the Sudoku, the weight matrix construction, and oscillator dynamics, without loss of generality. All principles, constraints, and mechanisms discussed for the 4×4 case scale naturally to the 9×9 case, or any size Sudoku. For the comparison with HNN, Sudokus of size 9×9 will be used.

A. Phase Mapping

The phase mapping is performed by evenly dividing the unit circle in a number steps equal to the number of digits required for the Sudoku. The phase mapping is done through the following equation:

$$\theta_{\text{digit}} = \frac{2\pi(n-1)}{N_d}, \quad (2)$$

where n is the current digit to be mapped, and N_d is the number of digits for the Sudoku version.

For the 4×4 case, the digits are mapped as follows:

$$\begin{aligned} 1 &\mapsto 0 \\ 2 &\mapsto \frac{\pi}{2} \\ 3 &\mapsto \pi \\ 4 &\mapsto \frac{3\pi}{2} \end{aligned}$$

Fig. 2 shows the phase mapping on the unit circle.

Using the introduced phase mapping, every Sudoku cell can be represented by a single oscillator. In this way, the ability of ONNs to encode information is fully exploited, instead of using multiple neurons per cell, as it has been done in [13], [14], one cell is directly represented by one oscillator, or neuron. By using only one neuron per cell, the neural network already becomes less complex than the network in [13], [14]. In general, this would mean that for an N_d -valued Sudoku, the complexity in the two cases would respectively be:

- ONN: $\mathcal{O}(N_d^2)$
- Hopfield Neural Network (HNN): $\mathcal{O}(N_d^3)$

To represent the network, one must define a weight matrix to generate the connections among oscillators. The weight matrix sizes are N_d^4 for the ONN, and N_d^6 for the HNN version [13], [14]. Specifically, for a standard 9×9 Sudoku, this work requires 81 neurons and a 81×81 weight matrix, whereas [13], [14] needs 729 neurons and a 729×729 weight matrix. A lower amount of neurons, and thus a lower number of connections, allows for easier implementation in hardware.

B. Graph Topology and Weight Mapping

In a graph representation for an ONN, one identifies the oscillators as the nodes of a graph, and the connections as the edges in a graph. The adjacency matrix of the graph directly maps to the weight matrix of the ONN. The graph topology needs to be designed in such a way that the natural dynamics of the oscillators solve the Sudoku, depending on which cells or nodes are known and which ones are unknown. There are three specific cases: an edge between two nodes with *known* values, an edge between two nodes with *unknown* values, and an edge between a node with a *known* value and a node with an *unknown* value. Each of these cases will be explained separately. Following this, a subgraph for each of the constraints of the Sudoku will be introduced, which are the row, column, and subgrid.

Firstly, for connections between two nodes with *known* values, there will be a bidirectional connection, where the weight J_{ij} of the connection is defined by the following relationship:

$$J_{ij}^{(\text{known})} = \begin{cases} e^{i(\theta_i - \theta_j)}, & \text{if } i \neq j, \\ 0, & \text{if } i = j. \end{cases} \quad (3)$$

Note that the coupling is *Hermitian* in general, which is a consequence of how the phases are put in relation to one another.

Secondly, for connections between two nodes with *unknown* values, there will be a bidirectional connection, where the weight J_{ij} of the connection is set to -1 . By choosing -1 , the distance between the phases is maximized. This is due to the need of having a repulsive relationship between such *unknown* oscillators, since these need to take on unique values, for the Sudoku to be solved.

Finally, for connections between a node with a *known* value and a node with an *unknown* value, there will be a unidirectional connection, with a value of -1 towards the *unknown* node. In this way, an *unknown* oscillator does not affect the *known* oscillator, but the *known* oscillator affects the *unknown* one.

The graphs for a subgrid, a row, and a column, can be seen in Figures 3, 4 and 5. It can be seen that the edges in each subgraph have been mapped as explained before. Additionally, one can subdivide symbolically the general Kuramoto dynamics given in Eq. (II) into one ODE for each puzzle constraint or subgraph. To make the notation for constraints less cumbersome, one can introduce a set notation, where the nodes involved in one constraint are included in the specific set. For each oscillator i these sets are the subgrid $\mathcal{S}_i$, row

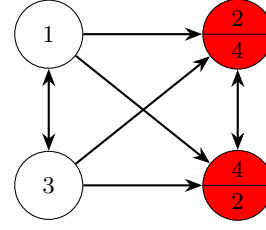


Fig. 3. Graph representation of the first Sudoku subgrid, dubbed the subgrid and denoted as $\mathcal{S}$.

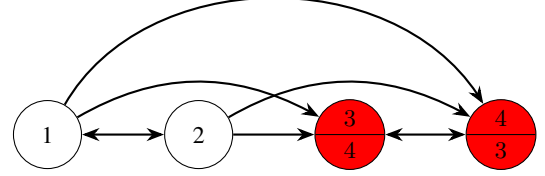


Fig. 4. Graph representation of the first Sudoku row, dubbed the row and denoted as $\mathcal{R}$.

$\mathcal{R}_i$, and column $\mathcal{C}_i$. The union of these sets is the set Ω_i of all constraints for that oscillator. Consequently, the subdivided Kuramoto dynamics are given by the following equations:

$$\dot{\theta}_i^{(\mathcal{S})} = - \sum_{j \in \mathcal{S}} J_{ij}^{(\mathcal{S})} \sin(\theta_j - \theta_i), \quad (4)$$

$$\dot{\theta}_i^{(\mathcal{R})} = - \sum_{j \in \mathcal{R}} J_{ij}^{(\mathcal{R})} \sin(\theta_j - \theta_i), \quad (5)$$

$$\dot{\theta}_i^{(\mathcal{C})} = - \sum_{j \in \mathcal{C}} J_{ij}^{(\mathcal{C})} \sin(\theta_j - \theta_i), \quad (6)$$

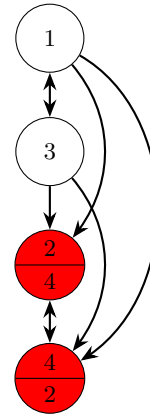


Fig. 5. Graph representation of the first Sudoku column, dubbed the column and denoted as $\mathcal{C}$.

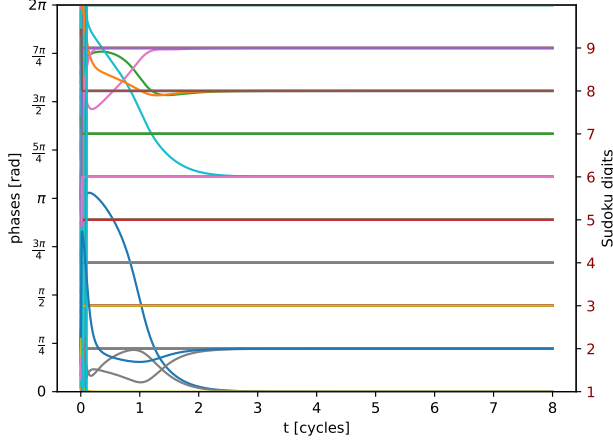


Fig. 6. Phase evolution of an ONN solving a 9×9 Sudoku with 10 unknown cells.

and the equation that defines the union is given by

$$\Omega_i = \mathcal{S}_i \cup \mathcal{R}_i \cup \mathcal{C}_i. \quad (7)$$

Finally, the union of all oscillator sets defines the complete puzzle as:

$$\Omega = \bigcup_{i=1}^N \Omega_i, \quad (8)$$

where N is the number of elements in the Sudoku, i.e. in a 4×4 Sudoku $N = 16$, and in a 9×9 Sudoku $N = 81$, in general $N = N_d^2$.

As a consequence, the equation for the total natural dynamics is then the linear combination of equations (4), (5), and (6), which can be succinctly written as:

$$\dot{\theta}_i = - \sum_{j \in \Omega_i} J_{ij} \sin(\theta_j - \theta_i). \quad (9)$$

For illustration, Fig. 6 shows a complete time trace of a randomized 9×9 Sudoku puzzle with 10 unknown cells.

C. Benchmarking Methodology

While the primary focus of this study is the evaluation of ONNs for solving Sudoku, the results are presented alongside those of a Hopfield network baseline, based on [13], [14], to provide a reference point and highlight the advantages of the proposed approach. The evaluation metrics include the number of correctly solved puzzles and the overall performance, defined as the percentage of correctly filled puzzles relative to the total number of puzzles. Simulations were conducted for puzzles with 5 to 35 unknown cells in increments of 5. For each specified number of unknown cells, that number of values was randomly removed from a fully solved Sudoku grid to generate the puzzle input. This removal process was performed using a randomized masking algorithm to simulate partial Sudoku inputs while ensuring a valid solution remained. For each case, 100 independent simulation runs were executed

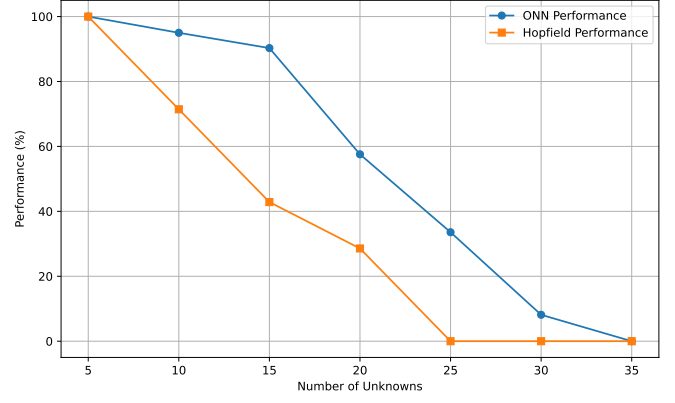


Fig. 7. Comparison of ONN and HNN performance across different numbers of unknowns.

to capture statistical robustness. For each simulation run, the phases were initialized randomly from a uniform distribution in the range $[0, 2\pi)$. A total of seven distinct puzzles were used in the evaluation for every level of difficulty: six were obtained from publicly available online sources [21], and the seventh was a benchmark puzzle taken from [14] to facilitate direct comparison with existing work. For each setting, multiple puzzles were solved, and the average performance was computed.

IV. RESULTS

Fig. 7 shows the results of the benchmark. As can be seen both the ONN and Hopfield networks achieve 100% correct solutions when solving puzzles with 5 unknown cells. However, as the number of unknowns increases, a performance gap between the two methods becomes evident. At 10 unknowns, the ONN maintains a high performance of 95%, while the Hopfield network drops significantly to 71.43%. This trend continues more drastically as puzzle complexity increases: with 15 unknowns, the ONN achieves over 90% performance, whereas the Hopfield network solves only 42.86% correctly. Beyond 20 unknowns, the Hopfield network's performance deteriorates sharply, reaching 0% performance for puzzles with 25 or more unknowns. In contrast, the ONN maintains partial effectiveness, achieving over 33% performance at 25 unknowns and still recovering some correct values at 30 unknowns. At 35 unknowns, neither network succeeds in solving any part of the puzzle correctly.

These results are further illustrated in Fig. 7, where the ONN's robustness against increasing puzzle complexity is visible compared to the sharp decline observed in the Hopfield network. Across the seven puzzles used for each difficulty level, the ONN consistently exceeded the Hopfield network in terms of performance. While individual puzzle performance varied slightly due to the random removal of values, the observed trend remained stable across all runs.

These results demonstrate that while both networks are capable of reliably solving Sudoku instances with a small number of unknown cells, their ability to scale to more

complex puzzles diverges significantly. The ONN maintains relatively high performance as puzzle difficulty increases, whereas the Hopfield network exhibits a rapid decline in performance beyond moderate difficulty. This difference in scalability becomes increasingly evident with higher levels of ambiguity and is further analyzed in detail in the following section.

V. DISCUSSION

The results presented in the previous section demonstrate that ONNs are capable of solving Sudoku puzzles with a high degree of performance, particularly when the number of unknown cells is moderate, meaning that the number of unknowns is less than 30. The performance of the ONN remains stable up to 15 unknowns, and it continues to produce partially correct solutions even in more ambiguous cases, such as puzzles with 25 or 30 missing values. **This resilience highlights the strength of the phase-based encoding and the ability of the network to dynamically enforce complex, distributed constraints over time.** The observed degradation in performance as the number of unknowns increases is expected given the reduced constraint information available to the network. However, the ONN continues to operate meaningfully even in cases where a baseline Hopfield network fails entirely.

While the ONN exhibits significantly improved performance over the baseline Hopfield network, particularly in the range of 5 to 20 unknowns, its ability to handle high-ambiguity puzzles remains limited. As the number of missing values increases beyond 25, the performance of the network drops sharply, and it completely fails to recover any correct entries at 35 unknowns. This performance ceiling suggests that while ONNs are effective for moderate-difficulty instances, they are not yet sufficient for solving highly incomplete puzzles without further enhancements. This limitation stems from the intrinsic nature of the Sudoku puzzle, where higher order logic is needed for a puzzle with a lower ratio of knowns to unknowns. **In this context, higher order logic refers to how sets of constraints interact and enforce phase dynamics on a global level, where predictions have to be made and corrected later if a contradiction in the solution is found.** In order to be able to solve puzzles that require higher order logic, **future research in developing an additional algorithm that analyses higher order interactions through the oscillator couplings is required.**

Nevertheless, the current results of the network, namely being able to solve Sudoku puzzles with up to 30 unknown values, could be improved. A solution for the moderate-difficulty puzzles, is to incorporate a checker-module, which checks the ratio of known cell values against the unknown cell values in the whole grid, and compares it to a threshold of clues. The ratio r is defined as:

$$r := \frac{N_{\text{unknowns}}}{N}, \quad (10)$$

quantifies the difficulty of the puzzle. In the checker module, the ratio may be computed, and then compared to a predefined threshold t , set by the user, leading to two distinct cases:

- **Case 1:** $r \geq t$ In this case, the ratio is higher than the threshold, and the module should perform N_{it} (e.g. we set $N_{\text{it}} = 100$) iterations of the algorithm. The unique solutions are collected, and the rest discarded.
- **Case 2:** $r < t$
In the second case, if the ratio is below the threshold value, the result should be accepted as the solution for the given puzzle, when correct.

The ability of ONNs to outperform a classical Hopfield network without external guidance, and to continue producing partial solutions even under sparse input conditions, indicates a promising foundation for further research in dynamic, constraint-based neural computation.

VI. CONCLUSION

This paper explored the use of Oscillatory Neural Networks (ONNs) for solving Sudoku puzzles, as a proof of concept, by leveraging their phase-based dynamics to enforce global constraints.

While the aim of the report was to solve Sudoku 9×9 grids, the methodology was presented through a 4×4 Sudoku grid to illustrate the constraint enforcement and oscillator dynamics in a manageable format. The same methodology is also applied to a 9×9 format by scaling the methods accordingly. The ONN architecture exploited the constraints by translating them into directed coupling strengths between oscillators, by having known values to guide synchronization to the right solution.

Experimental results showed that the ONN model demonstrated strong performance on puzzles with up to 20 unknown cells, correctly solving 57.57% of the runs at that point. Although performance degraded with increasing ambiguity, the network continued to produce some correct results at 25 unknowns before ultimately failing at 35. Compared to a baseline Hopfield network, the ONN showed greater robustness and adaptability, especially in moderately difficult puzzles, confirming its potential as a promising architecture for constraint satisfaction problems.

The discussion highlighted the ability of ONNs to operate without discrete logic, making use of the natural evolution of oscillator phases governed by the weight matrix and the Kuramoto synchronization model to solve constraints in a parallel and analog fashion. Although limitations were observed when increasing the puzzle difficulty, the ONN remained effective in delivering correct solution in cases in which the Hopfield Neural Network failed.

Overall, the study confirms that Oscillatory Neural Networks offer a promising alternative for solving constraint satisfaction problems such as Sudoku. The analog and parallel nature of the network makes it a promising option for future non-von Neumann computation. While challenges remain in scaling up the network for highly ambiguous cases, the proof of concept presented in this paper demonstrates the feasibility of ONNs as a computing paradigm for constraint-satisfaction problems using oscillator synchronization dynamics.

REFERENCES

- [1] J. Backus, “Can programming be liberated from the von neumann style? a functional style and its algebra of programs,” *Communications of the ACM*, vol. 21, no. 8, pp. 613–641, Aug. 1978. [Online]. Available: <https://doi.org/10.1145/359576.359579>
- [2] X. Zou, S. Xu, X. Chen *et al.*, “Breaking the von neumann bottleneck: architecture-level processing-in-memory technology,” *Science China Information Sciences*, vol. 64, no. 160404, Apr. 2021. [Online]. Available: <https://doi.org/10.1007/s11432-020-3227-1>
- [3] D. Ivanov, A. Chezhegov, A. Grunin, M. Kiselev, and D. Larionov, “Neuromorphic artificial intelligence systems,” 2022. [Online]. Available: <https://arxiv.org/abs/2205.13037>
- [4] D. Marković, A. Mizrahi, D. Querlioz, and J. Grollier, “Physics for neuromorphic computing,” *Nature Reviews Physics*, vol. 2, pp. 499–510, 2020.
- [5] A. Todri-Sanial, “Neuromorphic computing based on oscillatory neural networks,” in *SOPHIA Summit*, Sophia Antipolis, France, Nov. 2020, HAL Id: lirmm-03022129. [Online]. Available: <https://hal-lirmm.ccsd.cnrs.fr/lirmm-03022129>
- [6] A. Steane, “Quantum computing,” *Reports on Progress in Physics*, vol. 61, no. 2, p. 117, 1998.
- [7] M. Abernot, T. Gil, M. Jiménez, J. Núñez, M. J. Avellido, B. Linares-Barranco, T. Gonos, T. Hardelin, and A. Todri-Sanial, “Digital implementation of oscillatory neural network for image recognition applications,” *Frontiers in Neuroscience*, vol. 15, p. 713054, 2021.
- [8] E. M. Izhikevich, “Computing with oscillators,” *Neural Networks*, vol. 5255, pp. 1–30, 2000.
- [9] H. Simonis, “Sudoku as a constraint problem,” in *CP Workshop on modeling and reformulating Constraint Satisfaction Problems*, vol. 12. Citeseer Sitges, Spain, 2005, pp. 13–27.
- [10] P. Civioglu, “Backtracking search optimization algorithm for numerical optimization problems,” *Applied Mathematics and computation*, vol. 219, no. 15, pp. 8121–8144, 2013.
- [11] I. Lynce and J. Ouaknine, “Sudoku as a sat problem.” in *AI&M*. Citeseer, 2006.
- [12] T. Miyato, S. Löwe, A. Geiger, and M. Welling, “Artificial Kuramoto Oscillatory Neurons,” May 2025, arXiv:2410.13821 [cs] version: 3. [Online]. Available: <http://arxiv.org/abs/2410.13821>
- [13] J. J. Hopfield, “Searching for memories, Sudoku, implicit check-bits, and the iterative use of not-always-correct rapid neural computation,” Sep. 2006, arXiv:q-bio/0609006. [Online]. Available: <http://arxiv.org/abs/q-bio/0609006>
- [14] V. Mladenov, P. Karampelas, C. Pavlatos, and E. Zirintsis, “Solving sudoku puzzles by using hopfield neural networks,” *Proc. of ICACM*, vol. 11, pp. 174–179, 2011.
- [15] S. Gao, S. Xiang, Z. Song, Y. Han, and Y. Hao, “All-optical sudoku solver with photonic spiking neural network,” *Optics Communications*, vol. 495, p. 127068, 2021.
- [16] R. Pignari, V. Fra, E. Macii, and G. Urgese, “Efficient solution validation of constraint satisfaction problems on neuromorphic hardware: the case of Sudoku puzzles,” *IEEE Transactions on Artificial Intelligence*, pp. 1–11, 2025. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10887036>
- [17] H. Mori and Y. Kuramoto, *Dissipative Structures and Chaos*. Berlin, Heidelberg: Springer Berlin Heidelberg, 1998. [Online]. Available: <https://link.springer.com/10.1007/978-3-642-80376-5>
- [18] J. R. Dormand and P. J. Prince, “A family of embedded runge-kutta formulae,” *Journal of computational and applied mathematics*, vol. 6, no. 1, pp. 19–26, 1980.
- [19] L. F. Shampine, “Some practical runge-kutta formulas,” *Mathematics of computation*, vol. 46, no. 173, pp. 135–150, 1986.
- [20] E. Izhikevich and Y. Kuramoto, “Weakly coupled oscillators,” in *Encyclopedia of Mathematical Physics*, J.-P. Francoise, G. L. Naber, and T. S. Tsun, Eds. Oxford: Academic Press, 2006, pp. 448–453. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/B0125126662001061>
- [21] D. G. Radcliffe, “3 million sudoku puzzles with ratings,” 2020. [Online]. Available: <https://www.kaggle.com/dsv/1495975>